

Department of Electrical Engineering, University of Hawaii, May 27, 2026
EE 496 Final Report

Transformer Based Lossless Source Coding

EE 496 Final Report
Spring 2026

Department of Electrical Engineering
University of Hawaii at Mānoa

Dan Vu

May 27, 2026

Faculty Advisor: Professor Anders Høst-Madsen
Support: Yixin Zhang

Abstract

Høst-Madsen et al. [1] discussed that a learned source coder’s redundancy decays as $1/m$ in the number of training sequences, demonstrating that offline training can beat universal online coders once m is large enough. This paper empirically tests that prediction using a small transformer (minGPT-nano, $\sim 0.11\text{M}$ parameters) as a sequential probability estimator coupled with arithmetic coding, benchmarked against the Laplace estimator and Context Tree Weighting (CTW) across IID binary, Markov, and FSM sources. On IID sources, a general transformer trained across the full source distribution recovers the Laplace estimator from data alone. A specialized transformer crosses below the Laplace redundancy at around $m = 100$ training sequences. On structured sources the advantage of offline training widens. The specialized transformer beats both Laplace and CTW at all sequence lengths on the FSM source, with CTW closing the gap steadily. The results emphasize that learned and universal compression operate on the same mathematical engine of sequential prediction, and that offline knowledge of the source distribution is the mechanism by which a learned coder earns its advantage.

I. INTRODUCTION

A. Why Compression Matters

Compression comes in two flavors. Lossy compression discards information the receiver cannot perceive or does not need. JPEG drops high-frequency detail and MP3 removes inaudible components. Lossless compression discards nothing. Every binary bit of the original file can be recovered exactly. Lossless compression is required wherever one wrong bit corrupts the output. Some examples include executable code, legal records, medical images, cryptographic keys. There is no tolerance for approximation in those domains.

It turns out that the problem of lossless compression reduces to modeling the source distribution. A compressor assigns short codes to common patterns and long codes to rare ones. The better the probability model matches the true distribution, the shorter the average code. At scale this compounds: streaming platforms compress billions of files and digital archives grow by terabytes per day. A marginal improvement in the probability model shortens every symbol in every file. In this paper, we demonstrate the use of transformers as compressors, showing that data prediction is the same mathematical engine as data compression.

B. Entropy and Optimal Codes

In this paper, we test our compression models with IID data. We test with binary because it is a simple alphabet that can produce very specialized source distributions. Additionally, all modern digital data can be represented in binary.

A source emits binary symbols $x_1, x_2, \dots$ drawn from $\{0, 1\}$ from a distribution p . A code assigns a binary string to each symbol. We find an optimal coding scheme that minimizes the expected codelength $\mathbb{E}[L(X)]$. The optimal codelength assigned to symbol i which has a probability p_i is:

$$l_i^* = -\log_2 p_i. \quad (1)$$

The expected optimal codelength is therefore

$$H(X) = -\sum_x p(x) \log_2 p(x), \quad (2)$$

which is the Shannon entropy [2]. It is the minimum average bits per symbol achievable by any uniquely decodable code. No compression algorithm can do better than $H(X)$ on average.

C. Arithmetic Coding and the Model-Coder Separation

Arithmetic coding solves the integer constraint of Huffman coding. The key property is that the coder is agnostic to the probability model. Any sequential conditional estimator $\hat{p}(x_t | x_{1:t-1})$ plugs in directly. The total codelength is

$$L(x^l) = \sum_{t=1}^l -\log_2 \hat{p}(x_t | x_{1:t-1}). \quad (3)$$

With bits per symbol (BPS) calculated using

$$\text{BPS} = \frac{L(x^l)}{l} \quad (4)$$

When $\hat{p} = p_{\text{true}}$, BPS equals the entropy rate. Any deviation adds redundancy.

This separation is the mathematical engine of the paper. Compression quality equals estimation quality. The transformer, Laplace, and CTW are all probability estimators. Plugging any of them into (4) gives the codelength that arithmetic coding would achieve with that model. BPS is computable directly from a code length calculator that uses this arithmetic coder, with no real compressor required. Every experiment in this paper reduces to: which estimator predicts test sequences better?

D. Universal Source Coding

A universal coder requires no knowledge of the source distribution. The redundancy of a code with length function L for a source with parameter θ and sequences of length l is

$$R_l(L, \theta) = \frac{1}{l} \mathbb{E}_\theta[L(X^l)] - H_\theta(X). \quad (5)$$

This is the per-symbol gap above entropy.

This redundancy is minimized with a universal source coding algorithm that uses the *Laplace estimator* to estimate bit probabilities sequentially. At each position t it estimates

$$\hat{p}(x_{t+1} = 1 | x_{1:t}) = \frac{k + 1}{n + 2}, \quad (6)$$

where k is the count of ones seen and $n = t - 1$ is the total symbols seen. For bit prediction, this expression requires only running counts and no training.

CTW [3] blends Laplace estimates over all context lengths up to depth d , and is asymptotically more powerful than Laplace on sources with memory. Like Laplace, it requires no offline training, adapting instead to the test sequence during compression. To beat both CTW and Laplace estimation, a learned coder must exploit knowledge of the source distribution acquired through prior training.

II. EXPERIMENT DESIGN

A. System Overview

The full pipeline runs as follows. A source distribution generates training sequences. The transformer is trained on those sequences, testing on sequences drawn from the same source. BPS is computed by plugging transformer-generated bit predictions into equation (4). Laplace and CTW are evaluated on the same test sequences.

The paper reports six experiments. Experiment 1 trains a general transformer on sequences drawn uniformly from $(0, 1)$ and compares it against Laplace. Experiment 2 trains specialized transformers per source distribution and runs the same comparison. Experiment 2.5 trains transformers under non-uniform Beta priors and compares against the Bayes-optimal estimator for each prior. Experiment 3 sweeps the training set size m on a fixed IID source to measure redundancy decay. Experiment 4 benchmarks a

specialized transformer, the general IID transformer, Laplace, and CTW on a binary symmetric Markov chain, showing that the IID model’s in-context gains are modest compared to specialized training. Experiment 5 runs a Dirichlet sweep over four-state Markov models, comparing a specialized transformer, the general IID transformer, Laplace, and CTW. Experiment 6 benchmarks a specialized transformer against Laplace and CTW on a four-state FSM source.

B. The Transformer as Probability Estimator

The probability estimator is minGPT-nano [4], a scaled-down decoder-only transformer with approximately 0.11M parameters. The architecture has three transformer blocks, each with three causal self-attention heads and embedding dimension $d = 48$. The vocabulary is binary ($V = 2$), with token embeddings and learned positional embeddings both of dimension 48. The context length is 499 tokens, and therefore can be deployed for sequences up to 499 bits long.

The causal mask in each attention layer ensures that position t attends only to positions $1, \dots, t$, enforcing that no future symbol is visible. The input is the binary sequence $x_{1:t}$; the output at position t is a distribution over $\{0, 1\}$, from which we read off $\hat{p}(x_{t+1} = 1 \mid x_{1:t})$. A single forward pass produces all l conditional probabilities simultaneously.

Training is done with learning rate 10^{-4} and batch size 64. All models train for 1000 iterations on Apple MPS hardware.

C. Arithmetic Source Coding and CTW

CTW and arithmetic coding form a complete, modular compression system. The coder maintains an interval $[l, h) \subset [0, 1)$ and narrows it by $\hat{p}(x_t \mid x_{1:t-1})$ at each step. CTW supplies those probabilities by maintaining a binary tree of depth d , where each node holds a Laplace estimate conditioned on the corresponding context suffix. The FSM source used in Experiment 6 has deterministic transitions, so once CTW has seen enough symbols its tree captures the state dynamics exactly. The transformer plugs into the same arithmetic coding pipeline but gets its probabilities from a neural network trained offline. The key difference is that the transformer has already seen the FSM structure during training, while CTW must infer it from scratch on every test sequence.

III. EXPERIMENTS

The source parameter p denotes the Bernoulli bias. l denotes test sequence length. BPS values are means over 100 trials; error bars in all figures are one standard error.

A. Experiment 1: The General Transformer Learns the Laplace Estimator

The first experiment trains a single “general” transformer on sequences whose bias p is drawn uniformly from $(0, 1)$ at each training example. It sees the entire source distribution from $(0, 1)$ simultaneously and does not accumulate a strong prior for any single p . At test time it is evaluated on sequences from $(0, 1)$ and compared against Laplace.

Figure 1 shows the result. The general transformer tracks Laplace across all five p values at every sequence length. At $p = 0.3$, $l = 10$, Laplace achieves 0.980 BPS and the general transformer achieves 0.990 BPS. By $l = 499$ the gap narrows to 0.0005 BPS. The pattern holds across the full source distribution range.

The match is the finding. Laplace is Bayes-optimal under a uniform prior on p , so a transformer that converges to the same predictions has internalized an optimal Bayesian inference rule from data alone. The mixture training creates a uniform effective prior, and the transformer’s output distribution converges to the corresponding optimal predictor.

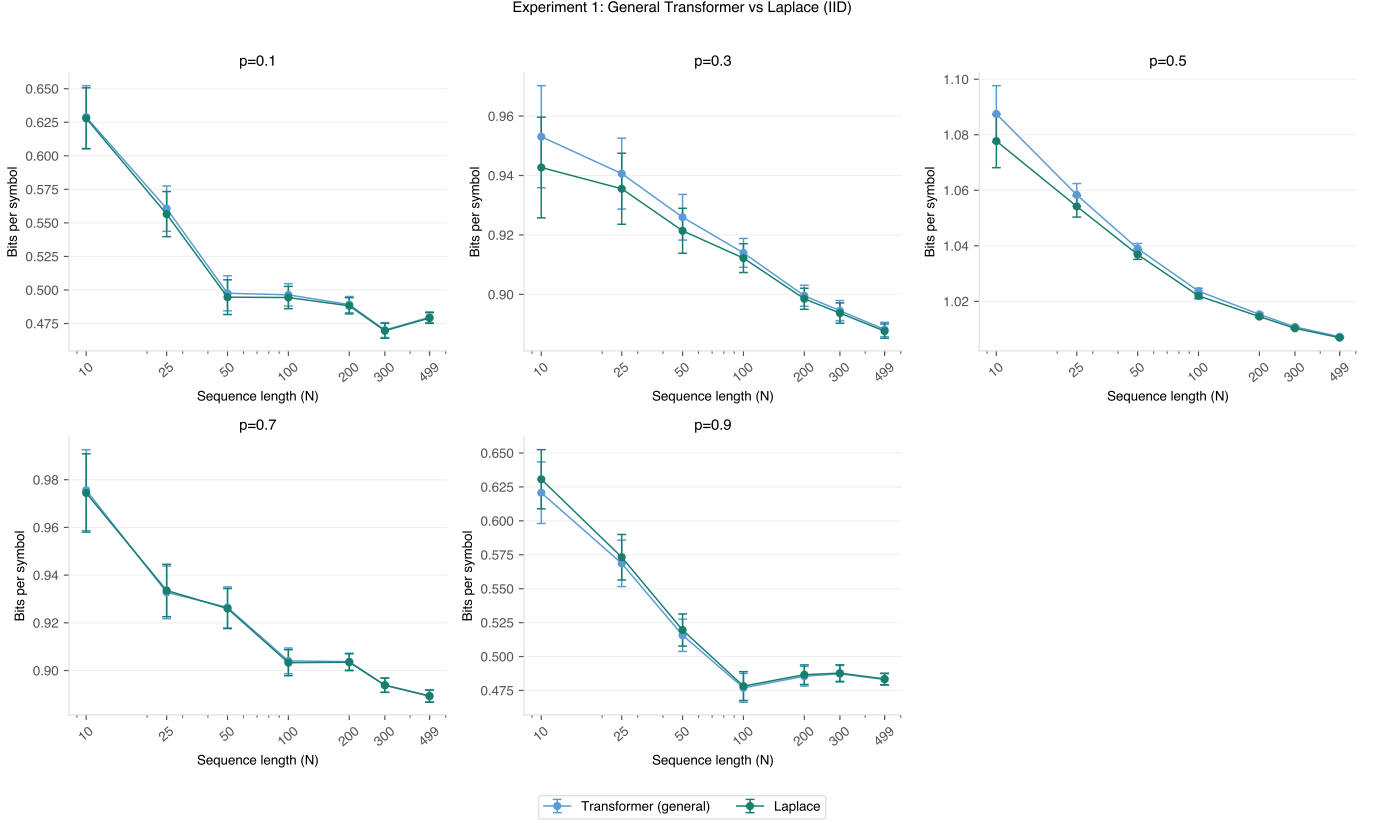


Figure 1. BPS vs. sequence length for the general transformer (blue) and Laplace (orange) across (0, 1). Entropy is shown as a dashed lower bound. Error bars are one standard error over 100 trials.

B. Experiment 2: A Specialized Transformer Beats Laplace at Short Sequences

A specialized transformer is trained on sequences from a single fixed bias. Two variants are evaluated: $p = 0.3$ and $p = 0.7$. Because training is concentrated on one distribution, the model develops a strong implicit prior for that source, similar in behavior to an optimal Bayesian estimator with a known prior.

At $p = 0.3$, $l = 10$: the specialized transformer achieves 0.894 BPS against Laplace’s 0.959 BPS, a gap of 0.065 bits per symbol. At $l = 499$ the gap collapses to 0.007 BPS. The $p = 0.7$ result is symmetric. This is because $H(0.3) = H(0.7)$ exactly: the binary entropy function is symmetric around $p = 0.5$, so the two sources have identical entropy.

The specialized model’s specialized training acts as a useful prior that gives it an advantage when the sequence is short. Laplace is Bayes-optimal under a uniform prior, and as l grows the empirical count dominates any prior term, so Laplace catches up. The transformer’s context window is also capped at 499 positions, so it cannot condition on arbitrarily long histories the way a classical count-based estimator can.

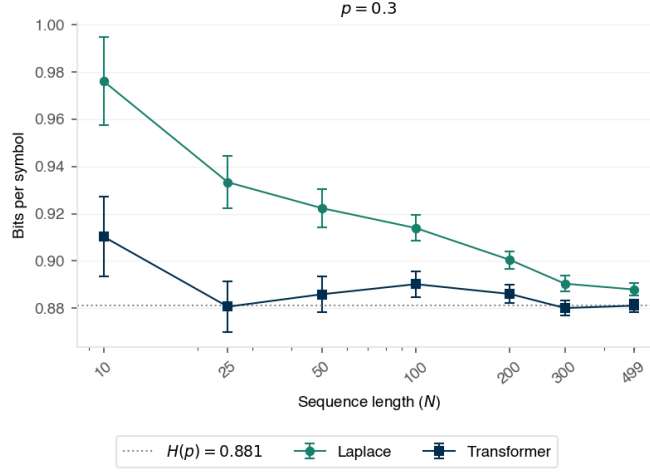


Figure 2. Absolute BPS for specialized ($p = 0.3$), general, Laplace, and entropy at $p = 0.3$.

C. Experiment 2.5: Prior Generalization to Beta

This experiment asks whether the specialized-beats-Laplace result is specific to the uniform prior or generalizes. Separate transformers are trained under five Beta priors: Beta(1, 1), Beta(0.5, 0.5), Beta(5, 5), Beta(5, 2), and Beta(2, 5). Each model is evaluated against Laplace and a Bayes-optimal estimator that uses the true prior analytically.

Figure 3 shows the result. Under Beta(1, 1), the transformer, Laplace, and estimator all align, consistent with Experiment 1. CTW performs poorly at small l across all non-uniform priors, never matching the transformer at short sequences.

The transformer adapts to the prior shape and consistently sits close to the estimator for every prior tested. The gap to the estimator is small but positive, which is expected since the transformer approximates the Bayes-optimal rule rather than computing it exactly.

Experiment 4: Beta Prior Generalization

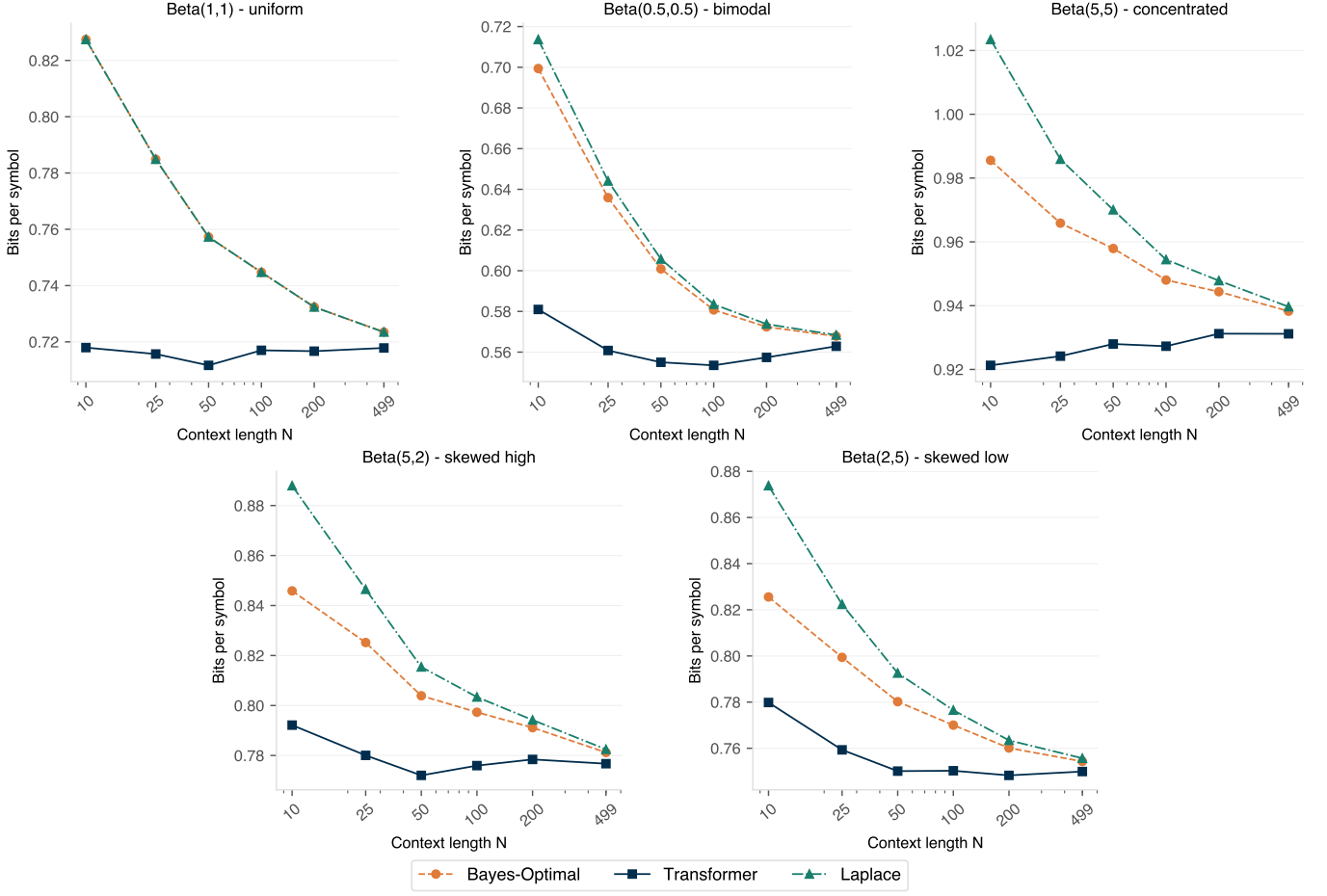


Figure 3. BPS vs. sequence length for transformer, Laplace, CTW, and the Bayes-optimal estimator under each Beta prior. The entropy floor for each prior is shown as a dashed line.

D. Experiment 3: Redundancy Scales as $1/m$

This experiment measures how transformer redundancy decays as training set size m grows. A fresh transformer is trained from scratch for each value of $m \in \{10, 25, 50, 75, 100, 150, 200, 300, 500, 1000\}$ on IID sequences with $p = 0.3$. All models are tested on sequences of length $l = 100$. Redundancy is BPS minus $H(0.3) = 0.881$ bits/symbol.

Figure 4 shows the result. At $m = 10$: redundancy 0.373 bits/symbol. At $m = 50$: 0.040. At $m = 100$: 0.021, already below the Laplace baseline of 0.022. By $m = 150$: 0.012, clearly dominant. The crossover at $m \approx 100$ is where the pretrain-learned prior becomes informative enough to outweigh Laplace’s parameter-free advantage.

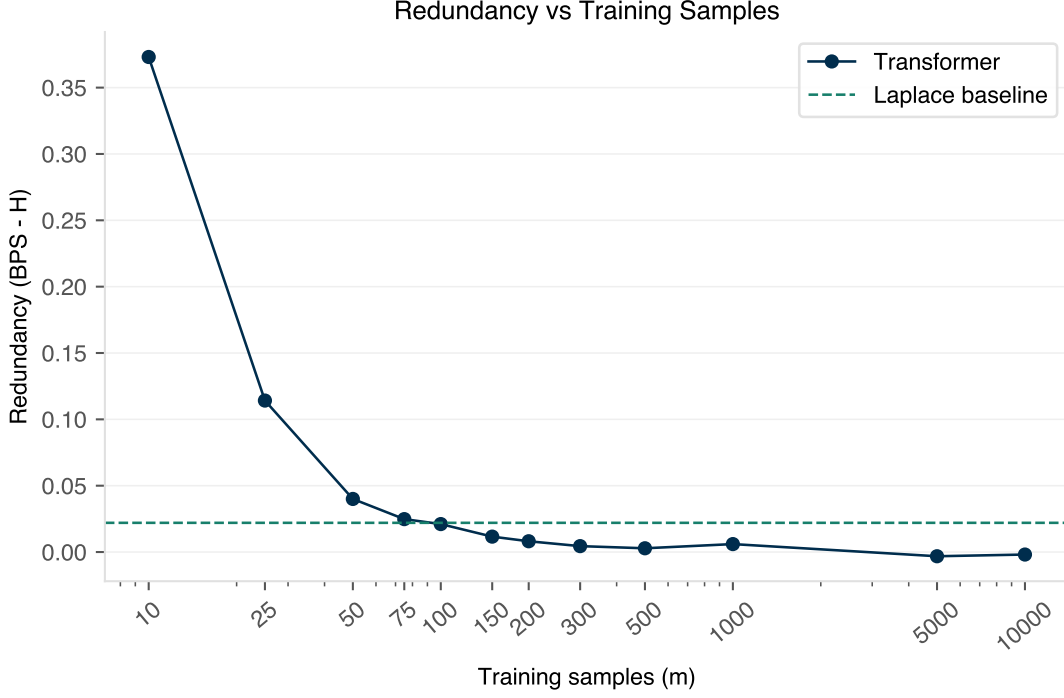


Figure 4. Transformer redundancy vs. training set size m on a log-log scale. The Laplace redundancy (flat at 0.022 bits/symbol) is shown as a dashed baseline.

E. Experiment 4: Binary Symmetric Markov Chain

This experiment runs four estimators on the binary symmetric Markov chain: a specialized transformer trained per p_{stay} , the general IID transformer from Experiment 1, Laplace, and CTW (depth 8).

Figure 5 shows the result. The first observation is that CTW and Laplace track closely across all p_{stay} values. At $p_{\text{stay}} = 0.99$, $l = 10$: Laplace 0.406 BPS, IID transformer 0.389 BPS, CTW 1.000 BPS. CTW has no full context window at $l = 10$ with depth 8, so it codes with flat 0.5 priors throughout. By $l = 50$ CTW recovers (0.346 BPS) and pulls ahead of Laplace, and by $l = 499$ CTW leads while the IID transformer has fallen behind both universal coders.

The specialized transformer is a different story. Trained on sequences from the same p_{stay} , it learns the transition structure offline and applies it immediately. At high p_{stay} it sits well below all three other estimators at every sequence length, and the gap does not close as l grows.

The IID model’s in-context gains over Laplace are real but small. CTW, a universal coder with no offline training, matches or exceeds the IID transformer at moderate to long sequence lengths. To consistently beat a universal coder on a structured source, specialized training is required.

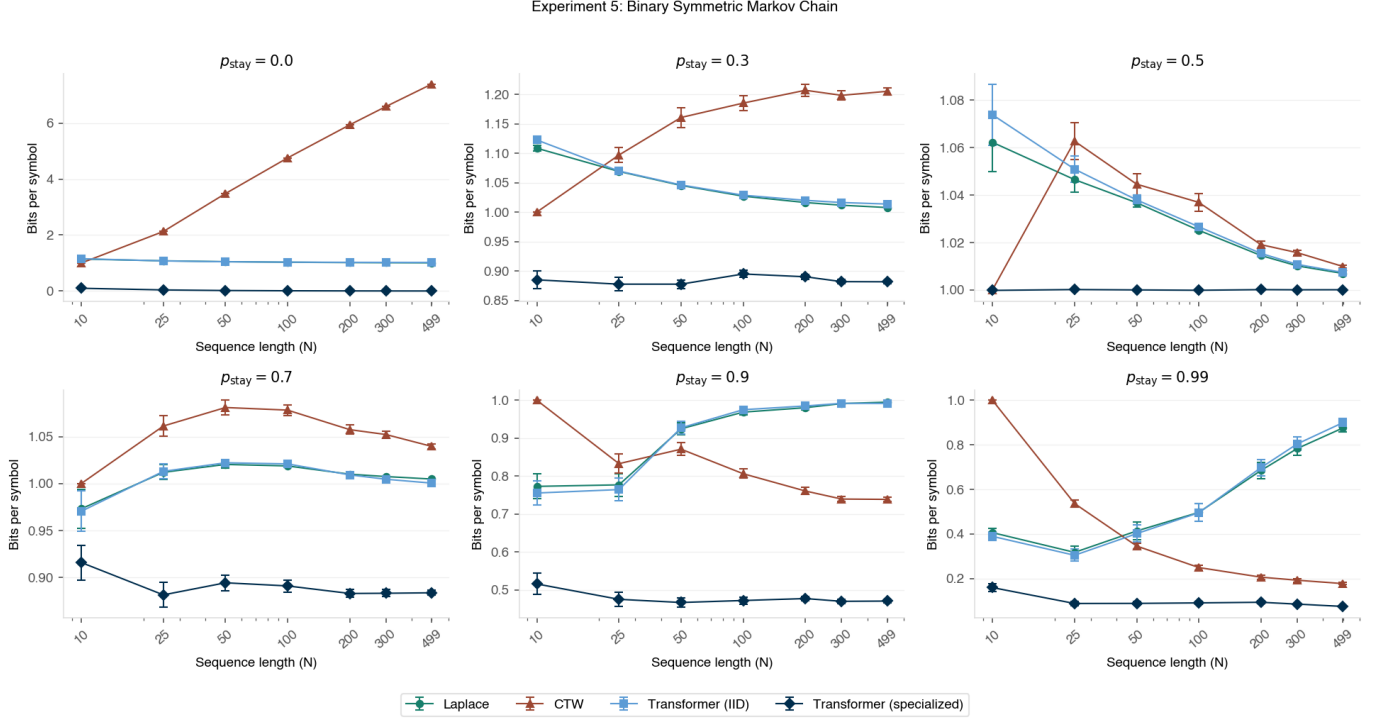


Figure 5. BPS vs. sequence length on the binary symmetric Markov chain. Specialized transformer (dark blue), IID transformer (light blue), CTW depth-8 (brown), Laplace (green). At $p_{\text{stay}} = 0.5$ the source is memoryless and all four estimators coincide.

F. Experiment 5: Dirichlet Markov Model Sweep

This experiment tests a four-state Markov model with emission probabilities $[0.1, 0.35, 0.65, 0.9]$ and a random transition matrix drawn per instance from a $\text{Dirichlet}(\alpha, \dots, \alpha)$ distribution. Larger α concentrates the transition matrix toward uniform (more randomness per step); smaller α concentrates it toward sparser transitions. Two sub-experiments are run for each $\alpha \in \{0.1, 0.5, 1.0, 2.0, 5.0, 10.0\}$: one with a specialized transformer trained per α , and one applying the general IID transformer.

Figure 6 shows the specialized transformer results. At $\alpha = 1.0$, $l = 100$: transformer 0.994 BPS, Laplace 1.009 BPS, CTW 1.022 BPS. The specialized transformer wins at every α tested.

Figure 7 shows the IID transformer result. It does not beat Laplace or CTW at any α . This is consistent with Experiment 4: even on the simpler binary symmetric Markov chain, the IID model’s in-context gains over universal coders were small and disappeared at longer sequences. On a four-state Markov model with Dirichlet-random transitions, the IID transformer offers nothing. Specialized training is required.

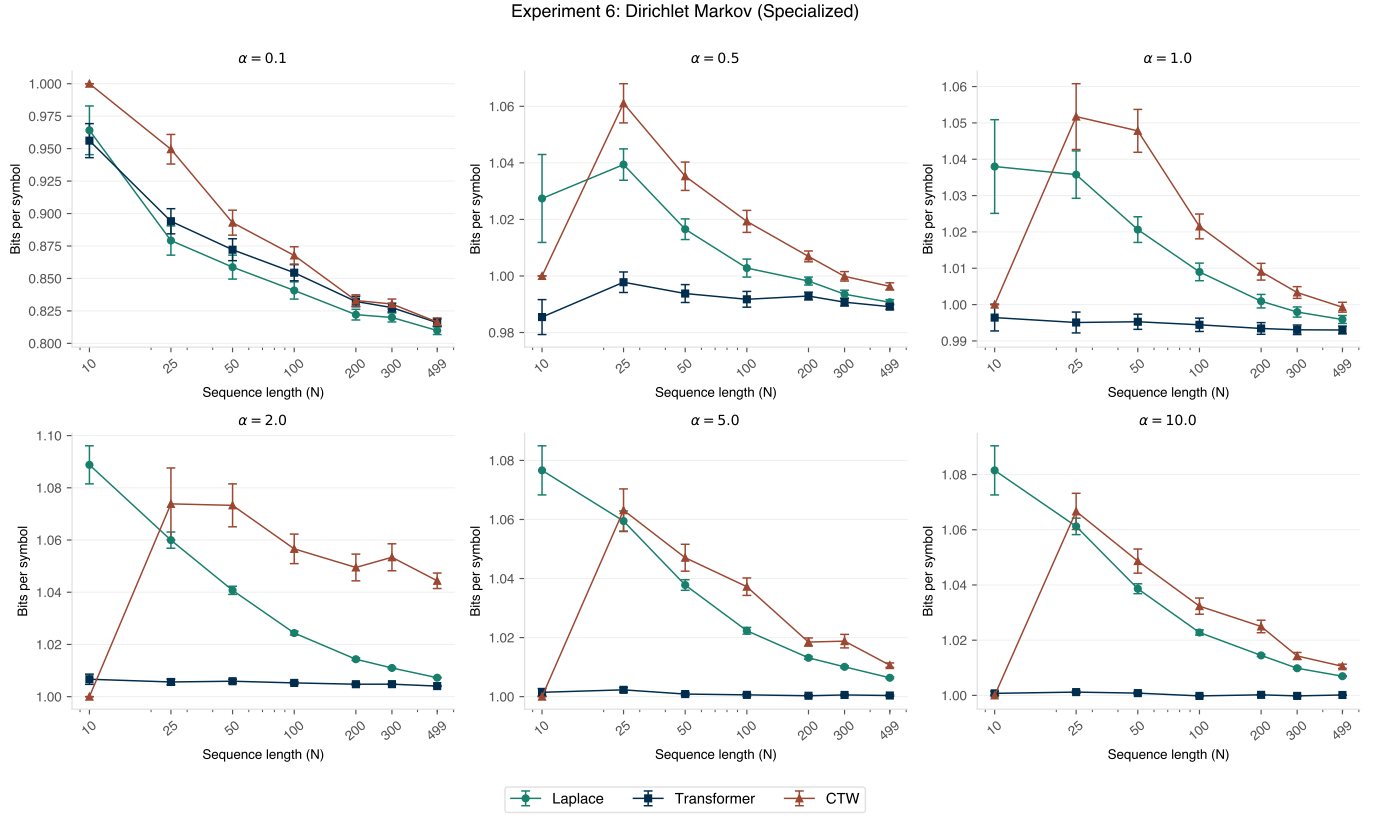


Figure 6. BPS vs. l for the specialized transformer (blue), Laplace (orange), and CTW depth-8 (green) at six Dirichlet concentration parameters α .

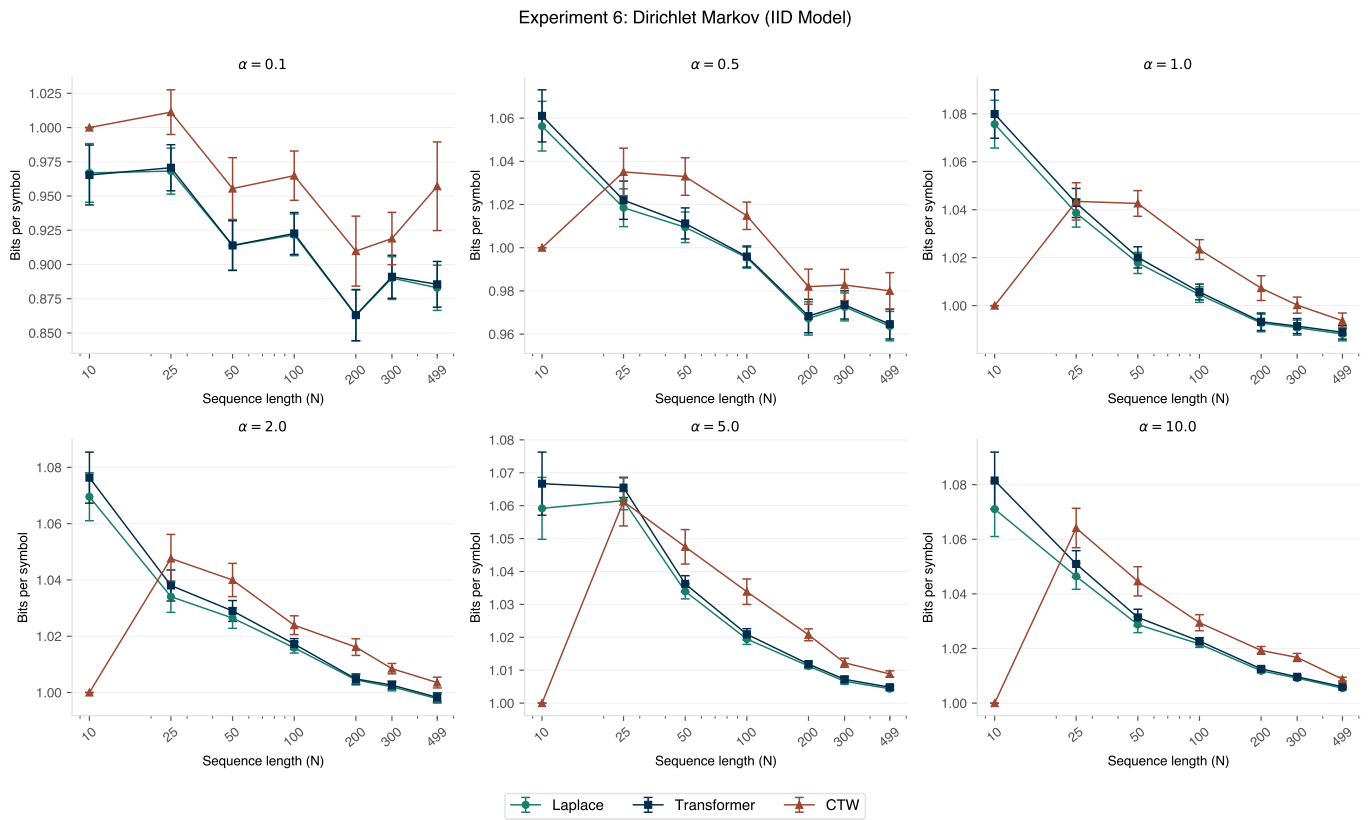


Figure 7. Same comparison with the general IID transformer in place of the specialized model. The IID transformer does not beat Laplace or CTW.

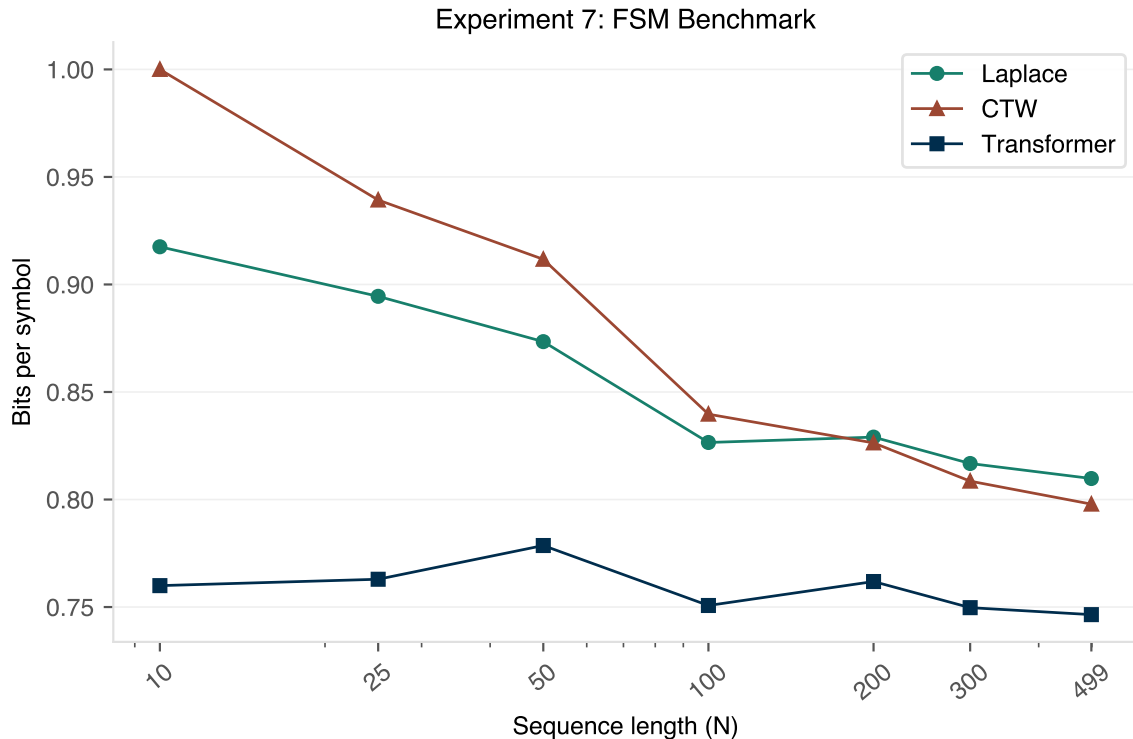


Figure 8. BPS vs. l for the specialized transformer, CTW depth-8, and Laplace on the four-state FSM source.

G. Experiment 6: FSM Source

The FSM source provides the clearest test of what offline training achieves against a proper universal coder. The three estimators here are the specialized transformer, Laplace, and CTW at depth 8. The FSM uses emission probabilities $[0.1, 0.35, 0.65, 0.9]$ and transition matrices

$$T_0 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \quad T_1 = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}, \quad (7)$$

where the next state depends on both the current state and the emitted bit.

Figure 8 shows the result. At $l = 10$: transformer 0.760 BPS, Laplace 0.918 BPS, CTW 1.000 BPS. At $l = 100$: transformer 0.751, CTW 0.840, Laplace 0.827. At $l = 499$: transformer 0.747, CTW 0.798, Laplace 0.810.

The transformer leads at every sequence length. CTW closes the gap steadily as l grows due to online learning. At short sequences the transformer’s offline training is worth roughly 0.24 BPS over CTW and 0.16 BPS over Laplace. That gap is the operational value of knowing the source distribution in advance.

IV. DISCUSSION

The transformer is both a frozen coder and an in-context predictor at once. Its weights are fixed after offline training on m sequences, and the distribution it applies to the test sequence is determined before any test bit arrives. On top of that, the transformer also reads the test sequence online, updating its prediction at each step. These two mechanisms are distinct, and the experiments show they contribute differently.

The pretrain-learned component is the prior encoded in the weights. When the transformer trains on sequences drawn from a specific source distribution, those weights encode a distribution over p concentrated near the true distribution. This happens entirely offline before the test sequence exists. The

in-context component is what the transformer does during coding: it conditions on the symbols it has already seen and refines its estimate of p step by step. Both contribute to the final BPS, but they operate on different aspects and require different sets of training data.

The general transformer’s convergence to Laplace demonstrates the in-context capability of transformers cleanly. The general transformer trains on a uniform distribution over p , which is exactly Laplace’s assumed prior. Same prior, same posterior updates, same predictions. The two models agree because the pretrain-learned component contributes nothing beyond what Laplace already assumes. What remains is pure in-context learning, and on i.i.d. sequences that is enough to match Laplace but not quite beat it.

The specialized transformer adds a useful prior. Its weights arrive at the test sequence already concentrated near the right p . At short l , the in-context component has little data to work with, and this pretrain-learned head start dominates. As l grows, empirical counts accumulate and the in-context component takes over for both the transformer and Laplace.

Experiment 2.5 shows the strongest version of this argument. The transformer does not just get a head start on Laplace. Trained under any Beta prior, it consistently tracks the Bayes-optimal estimator for that prior across all five families tested. This means the transformer is learning the Bayesian inference rule for the prior distribution.

Both CTW and Laplace have no pretrain-learned component. They start from scratch on every test sequence and rely entirely on in-context adaptation. On structured sources at moderate to long sequences, CTW matches or beats the IID transformer. The IID transformer’s pretrain-learned prior is not helpful on structured sources, and CTW’s in-context reconstruction eventually catches up.

The specialized transformer beats CTW on structured sources, but the mechanism is the pretrain-learned prior, not superior in-context adaptation. CTW must reconstruct source structure from the test sequence from scratch, but the specialized transformer already knows it. On the FSM source that offline knowledge is worth roughly 0.24 BPS over CTW at $l = 10$.

V. CONCLUSION

A small transformer can learn to perform lossless source coding at near-optimal rates. On IID sources, a general transformer recovers the Laplace estimator from training data alone, and a specialized transformer beats Laplace at short sequence lengths with the crossover around $m = 100$ training sequences. Redundancy decays at a rate consistent with the $1/m$ theoretical prediction. On structured sources, the IID transformer shows small in-context gains over Laplace on the binary symmetric Markov chain, but CTW achieves better performance with no offline training. The gains from Laplace-only comparisons do not hold against a proper universal coder baseline. Specialized training is what actually beats CTW, and the advantage grows substantially on four-state Markov and FSM sources.

The central takeaway is that learned compression and universal compression are not competing approaches. They operate on the same mathematical engine of prediction. The transformer earns its advantage by bringing offline knowledge of the source distribution that classical universal coders cannot access.

REFERENCES

- [1] A. Høst-Madsen, M. Zaeri Amirani, and N. P. Santhanam, “A bound for learning lossless source coding with online learning,” in *IEEE Int. Symp. Inf. Theory (ISIT)*, Taipei, Taiwan, Nov. 2024.
- [2] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Hoboken, NJ: Wiley-Interscience, 2006.
- [3] G. Schamberg, “Context tree weighting in Python,” <https://github.com/gabeschamberg/context-tree-weighting>, 2020, gitHub repository.
- [4] A. Karpathy, “minGPT: A minimal PyTorch re-implementation of the GPT training,” <https://github.com/karpathy/minGPT>, 2020, gitHub repository.